

Cheat sheet - 824 Node & Vite: Introduzione

Questa lezione fa da ponte tra gli esercizi JavaScript fatti finora e i progetti frontend moderni con React e Vite.

L'obiettivo principale non e' ancora imparare React in profondita', ma saper lavorare da terminale, capire cos'e' Node.js, usare npm, leggere un **package.json**, installare dipendenze e avviare un progetto React creato con Vite.

1. Mappa mentale della lezione

- **Terminale**: pannello di controllo testuale del computer. Da qui ci muoviamo tra cartelle, creiamo file, cancelliamo file e lanciamo programmi.
- **Shell**: programma che interpreta i comandi scritti nel terminale.
- **PowerShell**: shell moderna di Windows, quella piu' utile per la maggior parte della classe.
- **Bash / zsh**: shell comuni su macOS e Linux, da conoscere almeno come confronto.
- **File system**: struttura ad albero fatta di cartelle e file.
- **Node.js**: runtime che permette di eseguire JavaScript fuori dal browser.
- **npm**: gestore di pacchetti incluso con Node.
- **package.json**: file che descrive un progetto Node/npm.
- **Vite**: strumento che crea e avvia progetti frontend moderni, inclusi progetti React.
- **React**: libreria/framework frontend basata su componenti.

2. Terminale, shell e prompt su Windows

Il terminale e' la finestra in cui scriviamo comandi. La shell e' il programma che li legge ed esegue.

Su Windows useremo soprattutto **PowerShell**.

Esempio di prompt PowerShell:

```
PS C:\Users\Marco\code\boolean>
```

Significa:

- **PS**: stai usando PowerShell;
- **C:\Users\Marco\code\boolean**: cartella corrente;
- **>**: il terminale sta aspettando un comando.

Non serve ricordare tutti i dettagli del prompt: serve capire in quale cartella siamo e se il terminale sta aspettando un comando.

Nota per macOS/Linux: li' il prompt puo' essere diverso, per esempio **marco@macbook progetto %** oppure **utente@macchina:~/progetto\$**. In molti sistemi Unix **\$** indica un utente normale e **#** indica un utente amministratore/root.

3. File system: root, home e percorsi

Il file system e' una struttura ad albero.

Su Windows i dischi hanno lettere:

```
C:\  
D:\
```

Esempi di percorsi Windows:

```
C:\Users\Marco\Desktop  
C:\Users\Marco\code\boolean
```

La **home** e' la cartella personale dell'utente:

```
C:\Users\Marco
```

Il terminale spesso si apre proprio nella home o in una cartella vicina.

Nota per macOS/Linux: la radice del file system e' / , i percorsi usano / invece di \ , e la home si puo' abbreviare con ~ .

```
/Users/marco/Desktop  
/Users/marco/code/boolean  
~/Desktop
```

4. Percorsi assoluti e relativi

Un **percorso assoluto** parte dal disco o dalla radice del file system.

```
cd C:\Users\Marco\code\boolean
```

Nota per macOS/Linux: lo stesso percorso avrebbe una forma simile a questa:

```
cd /Users/marco/code/boolean
```

Un **percorso relativo** parte dalla cartella in cui ci troviamo adesso.

Esempi:

```
cd esercizio-1
cd ../altro-progetto
cd ./src
```

Simboli importanti:

Simbolo	Significato
.	cartella corrente
..	cartella padre
~	home dell'utente
C:\	disco principale su molti PC Windows
/	root su macOS/Linux, non e' la notazione principale Windows

Esempio:

```
cd ..
```

Significa: sali di una cartella.

5. Case sensitive e differenze macOS/Windows

Su Windows, nella pratica quotidiana, i percorsi sono spesso **case insensitive**:

```
Cartella
cartella
CARTELLA
```

vengono trattati come lo stesso nome.

Su Linux i percorsi sono **case sensitive**:

```
Cartella
cartella
```

possono essere due cartelle diverse.

Su macOS dipende dal formato del disco, ma conviene comunque comportarsi come se maiuscole e minuscole contassero.

Regola pratica per il corso: scegli nomi semplici, coerenti, senza spazi e senza accenti.

```
js-array-carousel  
react-blog  
vite-demo
```

Meglio evitare:

```
React Prova Finale  
esercizio con spazi  
Lezione Node e Vite
```

6. Comandi base da terminale Windows PowerShell

Dove sono?

```
pwd
```

Mostra il percorso della cartella corrente.

Vedere file e cartelle

```
ls  
dir  
ls -Force  
ls -Recurse
```

Significato:

- **ls**: mostra file e cartelle;
- **dir**: su PowerShell fa praticamente la stessa cosa di **ls**;
- **ls -Force**: mostra anche file nascosti;
- **ls -Recurse**: mostra ricorsivamente anche le sottocartelle.

Nota per macOS/Linux: spesso si usano **ls -la** per vedere dettagli e file nascosti, e **ls -R** per vedere sottocartelle ricorsivamente. Su Unix i file o cartelle che iniziano con **.** sono nascosti:

```
.git  
.env  
.DS_Store
```

Entrare in una cartella

```
cd nome-cartella
```

Tornare alla cartella padre:

```
cd ..
```

Andare alla home:

```
cd ~
```

Suggerimento: usa **Tab** per autocompletare nomi di file e cartelle. Se non completa, spesso il nome non esiste o hai sbagliato maiuscole/minuscole.

Creare una cartella

```
mkdir nome-cartella
```

Alias equivalente in PowerShell:

```
md nome-cartella
```

Creare un file

```
ni app.js
```

ni sta per **New-Item**.

Forma piu' esplicita:

```
New-Item app.js
```

Nota per macOS/Linux: il comando equivalente piu' comune e' **touch app.js**.

Scrivere testo in un file

```
echo "Hello" > file.txt
```

Attenzione: > sovrascrive il contenuto del file.

Per aggiungere testo in fondo al file:

```
echo "Altra riga" >> file.txt
```

Cancellare file

```
rm file.txt
```

oppure:

```
Remove-Item file.txt
```

Cancellare cartelle

```
rmdir nome-cartella
```

oppure:

```
Remove-Item nome-cartella -Recurse
```

Comando pericoloso:

```
Remove-Item * -Recurse
```

Cancella tutto quello che trova dentro la cartella corrente. Prima di usare comandi con `Remove-Item`, `rm`, `rmdir`, `*` o `-Recurse`, controlla sempre dove sei con `pwd` e cosa c'e' dentro con `ls`.

7. Jolly: asterisco

L'asterisco `*` indica "qualsiasi sequenza di caratteri".

Esempio:

```
cd *18
```

puo' entrare in una cartella che termina con 18, per esempio:

```
2018
```

Altro esempio:

```
rm *.txt
```

cancella tutti i file che finiscono con .txt.

Usalo con attenzione, soprattutto insieme a rm.

8. Aiuto sui comandi

In PowerShell puoi usare:

```
Get-Help nome-comando
```

Esempio:

```
Get-Help Remove-Item
```

Per vedere esempi:

```
Get-Help Remove-Item -Examples
```

Nota per macOS/Linux: spesso si usa `man nome-comando`, per esempio:

```
man ls
```

9. Vim: uscire se ci finisci dentro

A volte Git o il terminale possono aprire Vim. Se non lo conosci, puo' sembrare bloccato.

Per salvare e uscire:

```
:x
```

oppure:

```
ZZ
```

Per uscire senza salvare:

```
:q!
```

10. Node.js

Node.js e' un **runtime JavaScript** basato sul motore V8, lo stesso motore usato da Chrome per eseguire JavaScript.

Runtime significa: software che permette a un linguaggio di girare.

Finora abbiamo usato JavaScript soprattutto nel browser:

```
document.querySelector('h1');  
alert('Ciao');  
fetch('https://...');
```

Con Node possiamo eseguire JavaScript nel terminale:

```
// app.js  
console.log('Ciao da Node');
```

Eseguire il file:

```
node app.js
```

Controllare la versione installata:

```
node -v
```

Differenza importante:

- nel browser esistono **window** e **document**;
- in Node non esiste il DOM;
- in Node possiamo usare strumenti e pacchetti per lavorare su progetti, server, file e build.

11. npm

npm significa **Node Package Manager**.

Serve per:

- inizializzare progetti;
- installare librerie;
- salvare dipendenze;
- lanciare script;
- usare tool da riga di comando.

Controllare la versione:

```
npm -v
```

Inizializzare un progetto:

```
npm init
```

Inizializzare velocemente con valori di default:

```
npm init -y
```

Installare una libreria:

```
npm install nome-pacchetto
```

Abbreviazione:

```
npm i nome-pacchetto
```

Installare tutte le dipendenze di un progetto gia' esistente:

```
npm install
```

Questo comando legge `package.json` e `package-lock.json`, poi ricrea `node_modules/`.

12. package.json

`package.json` e' il file che descrive un progetto Node/npm.

Esempio semplificato:

```
{
  "name": "vite-react-demo",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "latest",
    "react-dom": "latest"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "latest",
    "vite": "latest"
  }
}
```

Campi importanti:

- **name**: nome del progetto.
- **private: true**: impedisce di pubblicare per sbaglio il progetto come pacchetto pubblico su npm.
- **version**: versione del progetto.
- **type: "module"**: abilita la sintassi moderna `import/export`.
- **scripts**: comandi rapidi eseguibili con npm.
- **dependencies**: librerie necessarie al progetto.
- **devDependencies**: strumenti utili solo durante sviluppo o build.

13. Dipendenze e devDependencies

Una **dipendenza** e' una libreria usata dal progetto.

Esempio:

```
npm install axios
```

Il pacchetto viene salvato in **dependencies**.

Una **dev dependency** e' uno strumento usato durante lo sviluppo, ma non e' parte diretta del codice finale dell'app.

Esempio:

```
npm install vite --save-dev
```

Il pacchetto viene salvato in **devDependencies**.

In un progetto Vite creato con scaffolding, molte cose sono gia' configurate automaticamente.

14. package-lock.json e node_modules

Quando installiamo pacchetti, npm crea o aggiorna:

```
package-lock.json  
node_modules/
```

package-lock.json contiene la fotografia precisa delle versioni installate, incluse le dipendenze delle dipendenze.

node_modules/ contiene fisicamente il codice dei pacchetti installati.

Regola Git:

- **package.json** va committato;
- **package-lock.json** va committato;
- **node_modules/** non va committata.

Esempio **.gitignore**:

```
node_modules/  
dist/  
.DS_Store
```

Se cloni un progetto e manca **node_modules/**, e' normale:

```
npm install
```

la rigenera.

15. Script npm

Gli script sono comandi salvati in **package.json**.

Esempio:

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  }
}
```

Per lanciarli:

```
npm run dev
npm run build
npm run preview
```

In un progetto Vite:

- `npm run dev`: avvia il server di sviluppo;
- `npm run build`: crea la versione pronta per produzione nella cartella `dist/`;
- `npm run preview`: prova localmente la build di produzione.

16. Moduli: CommonJS e ES Modules

In JavaScript esistono due stili principali per importare/esportare codice.

Stile CommonJS, piu' vecchio e molto usato in Node:

```
const fs = require('fs');
```

Stile ES Modules, moderno:

```
import React from 'react';
```

Con:

```
{
  "type": "module"
}
```

in `package.json`, Node tratta i file `.js` come moduli moderni e permette `import/export`.

Esempio export nominato:

```
// math.js
export function sum(a, b) {
  return a + b;
}
```

Esempio import nominato:

```
// app.js
import { sum } from './math.js';

console.log(sum(2, 3));
```

17. CLI

CLI significa **Command Line Interface**.

Una CLI e' un programma che si usa da terminale.

Esempi:

```
node -v
npm install
npm create vite@latest
git status
```

Molti tool moderni di sviluppo sono CLI: npm, Vite, Git, create-vite, TypeScript, ESLint e altri.

18. CDN vs npm

CDN significa **Content Delivery Network**.

Prima potevamo includere librerie nel browser con un link:

```
<script src="https://cdn.example.com/libreria.js"></script>
```

Con npm invece installiamo la libreria nel progetto:

```
npm install nome-libreria
```

e poi la importiamo:

```
import qualcosa from 'nome-libreria';
```

Nei progetti moderni useremo spesso npm perche' rende le dipendenze esplicite nel `package.json`.

19. Vite

Vite e' un tool per creare, avviare e buildare progetti frontend moderni.

Fa principalmente queste cose:

- crea lo scaffolding iniziale del progetto;
- avvia un dev server locale;
- aggiorna il browser quando salvi;
- supporta HMR, Hot Module Replacement;
- prepara la build finale;
- gestisce JSX, moduli e asset tramite plugin.

Scaffolding significa predisposizione iniziale dei file di progetto: cartelle, configurazioni, `package.json`, file sorgenti e boilerplate.

Creare un progetto React con Vite:

```
npm create vite@latest
```

Poi si sceglie:

```
Framework: React  
Variant: JavaScript
```

Entrare nella cartella:

```
cd nome-progetto
```

Installare dipendenze:

```
npm install
```

Avviare il progetto:

```
npm run dev
```

Vite mostrerà un URL simile a:

```
http://localhost:5173/
```

Di solito **5173** è la porta del sito in sviluppo.

Quando Vite è avviato, nel terminale puoi usare alcuni comandi rapidi. Per esempio:

```
o
```

apre il progetto nel browser.

20. Build, bundle, HMR

Bundle: output generato dai tool di build. In pratica i file sorgenti vengono trasformati, combinati e ottimizzati.

Build: processo che genera la versione finale del progetto:

```
npm run build
```

L'output finisce spesso in:

```
dist/
```

HMR significa Hot Module Replacement: quando modifichi un file, Vite aggiorna subito il browser, spesso senza ricaricare tutta la pagina.

Vite può anche minificare e ottimizzare i file in base a configurazione, plugin e librerie usate.

21. Struttura tipica di un progetto React con Vite

Esempio:

```
my-react-app/  
├── index.html  
├── package.json  
├── package-lock.json  
├── node_modules/  
├── public/  
└── src/  
    └── App.jsx
```

```
├─ main.jsx
└─ index.css
```

File importanti:

- **index.html**: pagina HTML di partenza.
- **src/main.jsx**: punto di ingresso JavaScript/React.
- **src/App.jsx**: primo componente principale.
- **package.json**: script e dipendenze.
- **node_modules/**: pacchetti installati, da non modificare e da non committare.

Differenza rispetto agli esercizi precedenti:

- non apriamo **index.html** con doppio click;
- avviamo il dev server con **npm run dev**;
- lavoriamo dentro **src/**;
- Vite si occupa di servire, compilare e aggiornare il progetto.

22. Primo contatto con React e JSX

React serve a costruire interfacce dividendo l'app in **componenti**.

Un componente e' un pezzo riutilizzabile di interfaccia.

Le app React sono organizzate come un albero di componenti:

```
App
├─ Header
├─ Main
│   └─ Card
│       └─ Card
└─ Footer
```

JSX e' una sintassi che assomiglia a HTML, ma vive dentro JavaScript.

Esempio:

```
const title = <h1>Ciao React</h1>;
```

Esempio componente:

```
function App() {
  return (
    <main>
      <h1>Ciao React</h1>
      <p>Primo progetto con Vite.</p>
    </main>
  )
}
```

```
);  
}  
  
export default App;
```

Nei file `.jsx` possiamo scrivere JSX. Vite si occupa di trasformarlo in JavaScript comprensibile dal browser.

Nota: dire che Vite "compila JSX in HTML" e' una semplificazione. Piu' precisamente, Vite e i suoi plugin trasformano i file sorgente in JavaScript/CSS/asset che il browser puo' eseguire e visualizzare.

23. Sequenza completa da ricordare

Creare e avviare un progetto React con Vite:

```
npm create vite@latest  
cd nome-progetto  
npm install  
npm run dev
```

Aprire l'URL indicato dal terminale, spesso:

```
http://localhost:5173/
```

Quando riprendi un progetto gia' creato:

```
cd nome-progetto  
npm install  
npm run dev
```

`npm install` serve solo se `node_modules/` manca o se sono cambiate le dipendenze.

24. Checklist anti-panico

Se il progetto non parte:

1. Sono nella cartella giusta?

```
pwd  
ls
```

2. Vedo `package.json`?

```
ls
```

3. Ho installato le dipendenze?

```
npm install
```

4. Quali script esistono?

```
cat package.json
```

5. Sto lanciando lo script giusto?

```
npm run dev
```

6. Sto aprendo l'URL mostrato da Vite?

```
http://localhost:5173/
```

25. Errori comuni

- **node non viene riconosciuto:** Node non e' installato o non e' nel PATH.
- **npm non viene riconosciuto:** npm non disponibile; di solito si risolve installando Node correttamente.
- **Missing script: dev:** nel `package.json` non esiste lo script `dev`, oppure non sei nella cartella giusta.
- **Cannot find module ...:** dipendenza mancante o `node_modules/` non installata; prova `npm install`.
- **Pagina bianca:** controlla la console del browser e il terminale dove gira Vite.
- **La pagina non si aggiorna:** controlla che `npm run dev` sia ancora attivo e che stai modificando i file giusti dentro `src/`.
- **Errore su import/export:** controlla path, graffe degli import nominati e `type: "module"` quando lavori con Node.
- **Ho cancellato cose con rm o Remove-Item:** prima di cancellare controlla sempre cartella corrente e contenuto.

26. Comandi da memorizzare

Terminale Windows PowerShell:

```
pwd
ls
ls -Force
cd nome-cartella
cd ..
mkdir nome-cartella
ni file.js
rm file.js
rmdir nome-cartella
Get-Help nome-comando
```

Node/npm:

```
node -v
npm -v
npm init -y
npm install
npm install nome-pacchetto
npm run dev
npm run build
```

Vite React:

```
npm create vite@latest
cd nome-progetto
npm install
npm run dev
```

27. Mini glossario

- **Bash/zsh**: shell comuni su macOS/Linux.
- **PowerShell**: shell moderna di Windows.
- **Path**: percorso di un file o cartella.
- **Path assoluto**: percorso completo dalla root o dal disco.
- **Path relativo**: percorso a partire dalla cartella corrente.
- **Root**: radice del file system.
- **Home**: cartella personale dell'utente.
- **Runtime**: ambiente che esegue codice.
- **CLI**: programma usabile da terminale.
- **npm**: gestore pacchetti di Node.
- **Package**: pacchetto installabile.
- **Dependency**: libreria necessaria al progetto.
- **Dev dependency**: libreria utile solo durante sviluppo/build.
- **Scaffolding**: creazione automatica della struttura iniziale di un progetto.
- **Boilerplate**: codice/file di partenza già predisposti.

- **Bundle:** output generato dal processo di build.
- **HMR:** aggiornamento rapido dei moduli durante lo sviluppo.
- **JSX:** sintassi simile a HTML scritta dentro JavaScript.
- **Componente:** pezzo riutilizzabile di interfaccia React.

28. Fonti utili

- Node.js Learn: <https://nodejs.org/learn/getting-started/introduction-to-nodejs>
- Node.js Learn - run scripts: <https://nodejs.org/learn/command-line/run-nodejs-scripts-from-the-command-line>
- npm Docs - package.json: <https://docs.npmjs.com/creating-a-package-json-file/>
- npm Docs - npm install: <https://docs.npmjs.com/cli/v11/commands/npm-install/>
- npm Docs - scripts: <https://docs.npmjs.com/cli/v11/using-npm/scripts/>
- Vite Guide: <https://vite.dev/guide/>
- React Docs: <https://react.dev/learn>
- MDN - JavaScript modules: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>